

TOTAL MODULES	CURRICULUM PART	CORE INNOVATIONS	TARGET ARCHITECTURE
76 Modules + 10 Practice Labs	Part 33 — Backend Architecture & Node.js	V8, Libuv, Streams, Buffers, Async I/O	REST APIs, Express, Microservices & Security

🌟 Advanced JavaScript for Node.js — সার্ভার-সাইড জাভাস্ক্রিপ্ট ও ফুলস্ট্যাক ব্যাকএন্ড

Node.js কোনো আলাদা প্রোগ্রামিং ভাষা নয়, বরং এটি একটি শক্তিশালী **JavaScript Runtime Environment** যা ব্রাউজারের বাইরে সার্ভারে জাভাস্ক্রিপ্ট চালানোর সুযোগ করে দেয়। V8 ইঞ্জিন, Libuv, ইভেন্ট লুপ, নন-ব্লকিং অ্যাসিঙ্ক্রোনাস I/O, প্রসেস অবজেক্ট, এনভায়রনমেন্ট ভারিয়েবল, কমনজেএস বনাম ইএস মডিউল, fs, path, os, events, Buffer, Streams এবং সিকিউর এক্সপ্রেস আর্কিটেকচার এই চ্যাপ্টারের মূল উপজীব্য।

🔥 1. V8 & Event Loop	🔥 2. process & env	🔥 3. CommonJS vs ESM	🔥 4. File System (fs)	🔥 5. path & os Modules
🔥 6. EventEmitter & once	🔥 7. Buffer & Streams	🔥 8. HTTP & Express APIs	🔥 9. Auth & Pool Security	🔥 10. Graceful Shutdown

Part 01 — Node.js Foundations, V8 Engine & Event Loop Architecture (39.1 – 39.7)

39.1 Node.js কী?

Node.js হলো এমন একটি runtime environment যার মাধ্যমে browser-এর বাইরে JavaScript চালানো যায়। আগে JavaScript মূলত:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Browser
↓
Chrome / Firefox / Edge
↓
JavaScript

Node.js-এর মাধ্যমে:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Computer / Server
↓
Node.js
↓
JavaScript

চালানো যায়।

39.2 Node.js দিয়ে কী করা যায়?

Node.js দিয়ে তৈরি করা যায়:

- Backend server
- REST API
- Authentication system
- Real-time application
- WebSocket server
- CLI tools
- File processing
- Automation
- Database applications
- Microservices
- Streaming applications

তোমার Full Stack Developer journey-তে:

🌟 FULL-STACK JAVASCRIPT SYSTEM TOPOLOGY

```
React
  ↓
Node.js
  ↓
Express.js
  ↓
Database
```

খুব গুরুত্বপূর্ণ architecture!

39.3 Browser JavaScript বনাম Node.js

Feature	Browser	Node.js
JavaScript	✓	✓
DOM	✓	✗
window	✓	✗
document	✓	✗
process	✗	✓
File System	সীমিত	✓
Server তৈরি	✗	✓
HTTP Server	Browser API	✓
fs module	✗	✓
path module	✗	✓
Database access	সাধারণত API দিয়ে	✓
npm ecosystem	ব্যবহার করা যায়	মূল ecosystem

♦ সহজভাবে:

Browser:

💡 Web page control করার জন্য JavaScript।

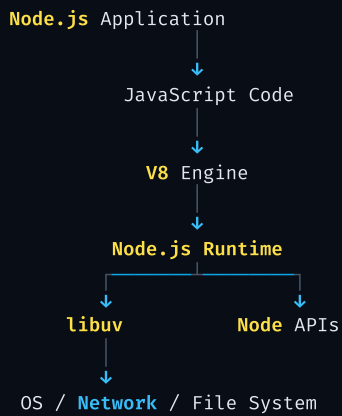
Node.js:

💡 Server/system-side কাজ করার জন্য JavaScript।

39.4 Node.js-এর ভিতরে কী আছে?

Simplified architecture:

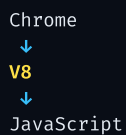
✦ NODE.JS INTERNAL ARCHITECTURE & EVENT LOOP



39.5 V8 Engine

V8 হলো Google-এর JavaScript engine। Chrome-এ:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM



Node.js-এ:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM



V8 JavaScript code execute করে।

39.6 Node.js Event Loop

Node.js-এর সবচেয়ে গুরুত্বপূর্ণ conceptগুলোর একটি হলো **Event Loop**। Node.js সাধারণত:

💡 Single JavaScript thread + asynchronous I/O model

ব্যবহার করে। উদাহরণ:

JAVASCRIPT Source

JAVASCRIPT

```
console.log("Start");

setTimeout(() => {
  console.log("Timer");
}, 0);

console.log("End");
```

Output:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Start
End
Timer

কারণ `setTimeout()` callback পরে execute হয়।

39.7 Node.js কেন দ্রুত?

Node.js-এর শক্তিশালী দিক হলো **non-blocking I/O**। ধরো server-কে file read করতে হবে:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Request
↓
File Read
↓
Waiting ...
```

পূরো JavaScript execution block না করে Node অন্য কাজ handle করতে পারে।

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Request A
↓
File I/O —————
Request B ← Node
Request C ← Node
                        ↓
                    File Result
```

এটি high-concurrency application-এর জন্য useful।

Part 02 — Process Object, CLI Arguments & Environment Variables (39.8 – 39.17)

39.8 process Object

Node.js-এ:

JAVASCRIPT Source

JAVASCRIPT

```
console.log(process);
```

process হলো Node.js process সম্পর্কিত information/control দেওয়ার global object।

39.9 process.version

JAVASCRIPT Source

JAVASCRIPT

```
console.log(process.version);
```

Example output:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
v24.x.x
```

তোমার নিজের system-এ installed Node version অনুযায়ী output আলাদা হবে।

39.10 process.platform

JAVASCRIPT Source

JAVASCRIPT

```
console.log(process.platform);
```

Windows হলে:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
win32
```

Linux হলে:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
linux
```

39.11 process.arch

JAVASCRIPT Source

JAVASCRIPT

```
console.log(process.arch);
```

Example:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

x64

CPU architecture জানা যায়।

39.12 process.argv

Command-line arguments পাওয়া যায়:

JAVASCRIPT Source

JAVASCRIPT

```
console.log(process.argv);
```

Run:

TERMINAL COMMAND

BASH

```
node app.js hello
```

তাহলে process.argv-তে command-line arguments থাকবে। Example:

JAVASCRIPT Source

JAVASCRIPT

```
const name = process.argv[2];  
  
console.log(`Hello ${name}`);
```

Run:

TERMINAL COMMAND

BASH

```
node app.js Ripon
```

Output:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Hello Ripon

♦ Real-life use

CLI application:

TERMINAL COMMAND

BASH

```
node create-user.js Ripon
```

39.13 Environment Variables

Server application-এ secret/configuration hard-code করা উচিত নয়। ❌:

JAVASCRIPT Source

JAVASCRIPT

```
const password = "mySecretPassword";
```

Better:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Environment Variable

↓

process.env

↓

Application

39.14 process.env

JAVASCRIPT Source

JAVASCRIPT

```
console.log(process.env.NODE_ENV);
```

Environment variable access করতে:

JAVASCRIPT Source

JAVASCRIPT

```
const port = process.env.PORT;
```

39.15 .env

সাধারণত:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
PORT=5000
DATABASE_URL=your_database_url
JWT_SECRET=your_secret
```

তারপর application-এ environment variables load করা হয়। Modern Node.js versions-এ অনেক ক্ষেত্রে:

TERMINAL COMMAND

BASH

```
node --env-file=.env server.js
```

ব্যবহার করা যায়। অথবা project-এ dotenv package ব্যবহার করা হয়।

JAVASCRIPT Source

JAVASCRIPT

```
import "dotenv/config";

console.log(process.env.PORT);
```

♦ 🔥 গুরুত্বপূর্ণ

Frontend-এর .env-এ রাখা value **secret ধরে নেওয়া যাবে না** যদি build process সেটি browser bundle-এ expose করে। Backend environment variables-এর ক্ষেত্রে secret রাখা যায়, তবে deployment platform-এর secret management ব্যবহার করাই ভালো।

39.16 __dirname

CommonJS-এ:

JAVASCRIPT Source

JAVASCRIPT

```
console.log(__dirname);
```

বর্তমান file-এর directory path পাওয়া যায়।

39.17 __filename

JAVASCRIPT Source

JAVASCRIPT

```
console.log(__filename);
```

বর্তমান file-এর সম্পূর্ণ path পাওয়া যায়।

♦ কিন্তু ES Modules-এ?

__dirname এবং __filename built-inভাবে থাকে না। ESM-এ:

JAVASCRIPT Source

JAVASCRIPT

```
import { fileURLToPath } from "node:url";
import path from "node:path";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
```

39.18 Node.js Modules

Node.js project-এ code আলাদা file/module-এ ভাগ করা হয়। দুটি প্রধান module system:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

CommonJS
ES Modules

39.19 CommonJS

পুরোনো/প্রচলিত Node.js module system। Export:

JAVASCRIPT Source

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}  
  
module.exports = add;
```

Import:

JAVASCRIPT Source

JAVASCRIPT

```
const add = require("./math");
```

39.20 Named CommonJS Export

JAVASCRIPT Source

JAVASCRIPT

```
function add(a, b) {  
  return a + b;  
}  
  
function subtract(a, b) {  
  return a - b;  
}  
  
module.exports = {  
  add,  
  subtract  
};
```

Import:

JAVASCRIPT Source

JAVASCRIPT

```
const { add, subtract } = require("./math");
```

39.21 ES Modules

Modern JavaScript style:

JAVASCRIPT Source

JAVASCRIPT

```
export function add(a, b) {  
  return a + b;  
}
```

Import:

JAVASCRIPT Source

JAVASCRIPT

```
import { add } from "./math.js";
```

Nodejs-এ ESM ব্যবহার করতে package.json-এ:

JSON PAYLOAD

JSON

```
{  
  "type": "module"  
}
```

দেওয়া যায়।

39.22 require() বনাম import

CommonJS	ES Modules
require()	import
module.exports	export
Node.js-এ traditional	Modern standard
__dirname available	import.meta.url
synchronous loading semantics	static module syntax

Modern full-stack projects-এ ESM অনেক বেশি দেখা যায়, তবে CommonJS এখনও গুরুত্বপূর্ণ।

39.23 Node.js Built-in Modules

Node.js অনেক built-in module দেয়। গুরুত্বপূর্ণ:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
fs  
path  
os  
url  
events  
http  
crypto  
stream  
buffer  
util  
process
```

Import:

JAVASCRIPT Source

JAVASCRIPT

```
import fs from "node:fs";
```

Modern Node.js-এ built-in modules-এর ক্ষেত্রে node: prefix ব্যবহার করা ভালো practice।

39.24 fs — File System

fs module file নিয়ে কাজ করতে ব্যবহার হয়।

JAVASCRIPT Source

JAVASCRIPT

```
import fs from "node:fs";

const data = fs.readFileSync("hello.txt", "utf8");

console.log(data);
```

এটি synchronous।

39.25 Async File Reading

Better server-side pattern:

JAVASCRIPT Source

JAVASCRIPT

```
import fs from "node:fs/promises";

const data = await fs.readFile(
  "hello.txt",
  "utf8"
);

console.log(data);
```

এটি asynchronous।

39.26 File Write

JAVASCRIPT Source

JAVASCRIPT

```
import fs from "node:fs/promises";

await fs.writeFile(
  "hello.txt",
  "Hello Ripon!"
);
```

ফাইল তৈরি/overwrite হবে।

39.27 Append

JAVASCRIPT Source

JAVASCRIPT

```
await fs.appendFile(
  "hello.txt",
  "\nNew line"
);
```

আগের content রেখে নতুন content যোগ করবে।

39.28 File Delete

JAVASCRIPT Source

JAVASCRIPT

```
await fs.unlink("hello.txt");
```

ফাইল delete করবে।

39.29 Directory Create

JAVASCRIPT Source

JAVASCRIPT

```
await fs.mkdir("uploads", {
  recursive: true
});
```

Folder তৈরি করবে।

39.30 path Module

File path handle করার জন্য:

JAVASCRIPT Source

JAVASCRIPT

```
import path from "node:path";
```

♦ Join

JAVASCRIPT Source

JAVASCRIPT

```
const filePath = path.join(
  "uploads",
  "images",
  "photo.jpg"
);

console.log(filePath);
```

OS অনুযায়ী correct separator ব্যবহার করবে।

39.31 path.basename()

JAVASCRIPT Source

JAVASCRIPT

```
const file = "/uploads/photo.jpg";

console.log(path.basename(file));
```

Output:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

photo.jpg

39.32 path.extname()

JAVASCRIPT Source

JAVASCRIPT

```
console.log(
  path.extname("photo.jpg")
);
```

Output:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

.jpg

39.33 path.dirname()

JAVASCRIPT Source

JAVASCRIPT

```
console.log(  
  path.dirname("/uploads/photo.jpg")  
);
```

Output:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
/uploads
```

39.34 os Module

Operating system information:

JAVASCRIPT Source

JAVASCRIPT

```
import os from "node:os";  
  
console.log(os.platform());  
console.log(os.arch());  
console.log(os.cpus().length);
```

Real-life:

💡 Server-এর CPU/core information জানতে।

39.35 url Module

JAVASCRIPT Source

JAVASCRIPT

```
import { URL } from "node:url";  
  
const url = new URL(  
  "https://example.com/products?page=2"  
);  
  
console.log(url.pathname);  
console.log(url.searchParams.get("page"));
```

Output:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
/products  
2
```

39.36 EventEmitter

Node.js-এর অন্যতম গুরুত্বপূর্ণ concept। Import:

JAVASCRIPT Source

JAVASCRIPT

```
import { EventEmitter } from "node:events";
```

Create:

JAVASCRIPT Source

JAVASCRIPT

```
const emitter = new EventEmitter();
```

Listener:

JAVASCRIPT Source

JAVASCRIPT

```
emitter.on("login", username => {
  console.log(`${username} logged in`);
});
```

Event trigger:

JAVASCRIPT Source

JAVASCRIPT

```
emitter.emit("login", "Ripon");
```

Output:

🔥 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Ripon logged in

39.37 Real-Life EventEmitter

ধরো application-এ:

🔥 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```

User Registration
  ↓
"userRegistered"
  ↓
├── Email
├── Log
└── Analytics
```

একটি event-এর জন্য বিভিন্ন listener কাজ করতে পারে।

39.38 once()

কোনো event একবারের বেশি handle করতে না চাইলে:

JAVASCRIPT Source

JAVASCRIPT

```
emitter.once("startup", () => {
  console.log("Server started");
});
```

একবার execute হওয়ার পর listener remove হয়ে যায়।

39.39 Buffer

Node.js-এ binary data handle করার জন্য Buffer ব্যবহৃত হয়।

JAVASCRIPT Source

JAVASCRIPT

```
const buffer = Buffer.from("Hello");  
  
console.log(buffer);
```

এটি bytes representation তৈরি করবে। আবার:

JAVASCRIPT Source

JAVASCRIPT

```
console.log(buffer.toString());
```

Output:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Hello

♦ কোথায় ব্যবহার হয়?

- File
- Image
- Audio
- Video
- TCP
- Network data

39.40 Streams

বড় data একবারে memory-তে না এনে ছোট ছোট chunk হিসেবে process করার জন্য Streams ব্যবহার করা হয়। ধরো 5GB video! ❌ পুরো file memory-তে load করা:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

5GB
↓
RAM

সমস্যা হতে পারে। Stream:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

5GB File
↓
Chunk
Chunk
Chunk
Chunk
Chunk
↓
Output

এটি অনেক efficient।

39.41 Stream Types

চারটি প্রধান ধরনের stream:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Readable
Writable
Duplex
Transform

39.42 Readable Stream

যেখান থেকে data পড়া যায়। উদাহরণ:

JAVASCRIPT Source

JAVASCRIPT

```
import fs from "node:fs";

const stream = fs.createReadStream(
  "large.txt",
  "utf8"
);

stream.on("data", chunk => {
  console.log(chunk);
});
```

39.43 Writable Stream

যেখানে data লেখা যায়।

JAVASCRIPT Source

JAVASCRIPT

```
const stream = fs.createWriteStream(
  "output.txt"
);

stream.write("Hello");
stream.write("\nWorld");

stream.end();
```

39.44 pipe()

Readable stream → Writable stream:

JAVASCRIPT Source

JAVASCRIPT

```
const readStream =
  fs.createReadStream("input.txt");

const writeStream =
  fs.createWriteStream("output.txt");

readStream.pipe(writeStream);
```

Concept:

🌟 STREAM DATA FLOW PIPELINE

```
input.txt
  ↓
Readable Stream
  ↓
pipe()
  ↓
Writable Stream
  ↓
output.txt
```

🔥 Node.js-এ Streams-এর core concept।

39.45 Transform Stream

Data read করার সময় transform করা যায়। উদাহরণ:

★ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Input
↓
Transform
↓
Output
```

Compression tools যেমন gzip-এর ক্ষেত্রে এই concept ব্যবহৃত হয়।

Part 06 — Non-blocking Async I/O & Robust Error Handling (39.46 – 39.51)

39.46 Node.js Async I/O

Node.js-এ asynchronous API খুব গুরুত্বপূর্ণ। ❌ Blocking:

JAVASCRIPT Source

JAVASCRIPT

```
const data = fs.readFileSync("file.txt");
```

এটি synchronous। Better:

JAVASCRIPT Source

JAVASCRIPT

```
const data = await fs.readFile("file.txt");
```

Server application-এ non-blocking operations সাধারণত preferred।

39.47 Callback

পুরোনো Node.js code-এ callback ধরুর দেখা যায়।

JAVASCRIPT Source

JAVASCRIPT

```
fs.readFile(
  "hello.txt",
  "utf8",
  (error, data) => {
    if (error) {
      console.error(error);
      return;
    }

    console.log(data);
  }
);
```

39.48 Callback Hell

অনেক nested callback:

JAVASCRIPT Source

JAVASCRIPT

```
doSomething(() => {
  doSomethingElse(() => {
    anotherTask(() => {
      finalTask(() => {
        console.log("Done");
      });
    });
  });
});
```

এটি maintain করা কঠিন। Modern solution:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Promise
↓
async/await
```

39.49 Async/Await

JAVASCRIPT Source

JAVASCRIPT

```
async function readFile() {
  try {
    const data = await fs.readFile(
      "hello.txt",
      "utf8"
    );

    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

Code দেখতে synchronous-এর মতো হলেও operation asynchronous।

39.50 Error Handling

Node.js application-এ error properly handle করা অত্যন্ত গুরুত্বপূর্ণ।

JAVASCRIPT Source

JAVASCRIPT

```
try {
  const data = await fs.readFile(
    "missing.txt",
    "utf8"
  );
} catch (error) {
  console.error(
    "File error:",
    error.message
  );
}
```

39.51 Custom Error

নিজের Error class তৈরি করা যায়:

JAVASCRIPT Source

JAVASCRIPT

```
class AppError extends Error {
  constructor(message, statusCode) {
    super(message);
    this.statusCode = statusCode;
  }
}
```

ব্যবহার:

JAVASCRIPT Source

JAVASCRIPT

```
throw new AppError(
  "User not found",
  404
);
```

Express backend-এ এই pattern খুব usefull

Part 07 — HTTP Server, REST APIs & Express Middleware Mechanics (39.52 – 39.59)

39.52 HTTP Server

Node.js দিয়ে directly HTTP server তৈরি করা যায়।

JAVASCRIPT Source

JAVASCRIPT

```
import http from "node:http";

const server = http.createServer(
  (req, res) => {
    res.writeHead(200, {
      "Content-Type": "text/plain"
    });

    res.end("Hello World");
  }
);

server.listen(3000, () => {
  console.log(
    "Server running on port 3000"
  );
});
```

Browser:

🚀 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

http://localhost:3000

Output:

🚀 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Hello World

39.53 Request এবং Response

JAVASCRIPT Source

JAVASCRIPT

```
http.createServer((req, res) => {  
  console.log(req.method);  
  console.log(req.url);  
  
  res.end("Hello");  
});
```

req:

💡 Client কী request করেছে।

res:

💡 Server client-কে কী response দেবে।

39.54 HTTP Methods

Backend development-৯:

Method	কাজ
GET	Data নেওয়া
POST	নতুন data তৈরি
PUT	পুরো resource update
PATCH	আংশিক update
DELETE	Delete

Example:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
GET /users  
POST /users  
GET /users/10  
PATCH /users/10  
DELETE /users/10
```

39.55 HTTP Status Codes

♦ 2xx

Success:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
200 OK
201 Created
204 No Content
```

♦ 4xx

Client-side/request problem:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Content
```

♦ 5xx

Server problem:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
```

39.56 JSON API

Server:

JAVASCRIPT Source

JAVASCRIPT

```
const data = {
  name: "Ripon",
  age: 23
};

res.setHeader(
  "Content-Type",
  "application/json"
);

res.end(JSON.stringify(data));
```

Client পাঠবে:

JSON PAYLOAD

JSON

```
{
  "name": "Ripon",
  "age": 23
}
```

39.57 Node.js + Express

Real-world project-এ raw Node HTTP server-এর পরিবর্তে সাধারণত Express/Fastify/NestJS-এর মতো framework ব্যবহার করা হয়। Typical architecture:

✦ FULL-STACK JAVASCRIPT SYSTEM TOPOLOGY

```
React
↓
HTTP Request
↓
Express / Node.js
↓
Router
↓
Middleware
↓
Controller
↓
Service
↓
Database
```

39.58 Middleware Concept

Middleware হলো request-response lifecycle-এর মাঝখানে চলা function। Concept:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Request
↓
Middleware
↓
Authentication
↓
Controller
↓
Response
```

Example Express:

JAVASCRIPT Source

JAVASCRIPT

```
app.use((req, res, next) => {
  console.log(req.method);
  next();
});
```

39.59 Authentication Middleware

JAVASCRIPT Source

JAVASCRIPT

```
function auth(req, res, next) {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).json({
      message: "Unauthorized"
    });
  }

  next();
}
```

এখানে middleware protected route-এর আগে authentication check করতে পারে।

39.60 Node.js + Database

Node.js নিজে database নয়। এটি database-এর সাথে communicate করে।

✦ FULL-STACK JAVASCRIPT SYSTEM TOPOLOGY

```

React
↓
Node.js / Express
↓
Database Driver / ORM
↓
Database
  
```

Possible databases:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```

MongoDB
PostgreSQL
MySQL
MariaDB
SQLite
Redis
  
```

তোমার stack-এর জন্য:

✦ FULL-STACK JAVASCRIPT SYSTEM TOPOLOGY

```

React
↓
Node.js
↓
Express
↓
MongoDB / PostgreSQL
  
```

খুব practical combination।

39.61 Database Connection Pool

অনেক request এলে প্রত্যেক request-এর জন্য নতুন database connection তৈরি করা inefficient হতে পারে। Pool:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```

Connection Pool
├── Connection 1
├── Connection 2
├── Connection 3
└── Connection 4
  
```

প্রয়োজনে connection নেওয়া হয় এবং পরে release করা হয়।

39.62 Node.js Security

Backend-এর জন্য গুরুত্বপূর্ণ:

♦ Secrets

✗:

JAVASCRIPT Source

JAVASCRIPT

```
const jwtSecret = "123456";
```

Better:

JAVASCRIPT Source

JAVASCRIPT

```
const jwtSecret = process.env.JWT_SECRET;
```

♦ Input Validation

Client-এর input blindly trust করবে না।

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Client Input
↓
Validation
↓
Sanitization (যেখানে প্রয়োজন)
↓
Business Logic
↓
Database
```

♦ এছাড়াও:

- Authentication
- Authorization
- Rate limiting
- HTTPS
- Secure headers
- CORS configuration
- Password hashing
- Dependency security
- SQL/NoSQL injection prevention

39.63 Password কখনো Plain Text নয়

✗:

JAVASCRIPT Source

JAVASCRIPT

```
password: "mypassword123"
```

Database-এ plain password রাখা উচিত নয়। Password hashing-এর জন্য সাধারণত:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Argon2
bcrypt
scrypt
```

এর মতো password hashing algorithms ব্যবহৃত হয়।

39.64 Node.js Performance

Node.js ভালো performance দিতে পারে, কিন্তু JavaScript thread-এ heavy CPU কাজ করলে সমস্যা হতে পারে। যেমন:

JAVASCRIPT Source

JAVASCRIPT

```
for (;;) {  
  // CPU heavy work  
}
```

এটি event loop block করতে পারে। ফলে:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Request A
Request B
Request C

সব আটকে যেতে পারে।

39.65 CPU-heavy কাজের Solution

প্রয়োজনে:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Worker Threads
Child Processes
Separate **Service**
Queue / Background Worker

ব্যবহার করা যায়। Worker Threads:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

Main Thread
├── Worker 1
└── Worker 2

39.66 Graceful Shutdown

Production server বন্ধ করার সময় open resources properly close করা উচিত।

JAVASCRIPT Source

JAVASCRIPT

```
process.on("SIGTERM", async () => {  
  console.log("Shutting down...");  
  
  // close database  
  // close server  
  // cleanup resources  
  
  process.exit(0);  
});
```

এটি graceful shutdown-এর basic ideal

39.67 Debugging

Simple:

JAVASCRIPT Source

JAVASCRIPT

```
console.log()
console.error()
console.warn()
```

Node debugging:

TERMINAL COMMAND

BASH

```
node inspect server.js
```

VS Code debugger-ও ব্যবহার করা যায়।

39.68 Logging

Production application-এ শুধু:

JAVASCRIPT Source

JAVASCRIPT

```
console.log()
```

এর উপর নির্ভর না করে structured logging ব্যবহার করা যায়। যেমন:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
timestamp
level
message
requestId
userId
```

এর মাধ্যমে production issue trace করা সহজ হয়।

39.69 Node.js Project Structure

একটি সাধারণ Express project:

🌟 NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
project/
├── src/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── services/
│   ├── utils/
│   └── server.js
├── .env
├── .env.example
├── .gitignore
├── package.json
└── package-lock.json
```

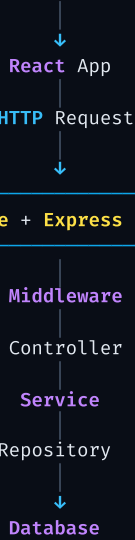
এটি একমাত্র correct structure নয়; project অনুযায়ী structure পরিবর্তিত হতে পারে।

39.70 Full Stack Architecture

তুমি যেহেতু Full Stack Developer হতে চাচ্ছে, এই architecture ভালোভাবে বুঝবে:

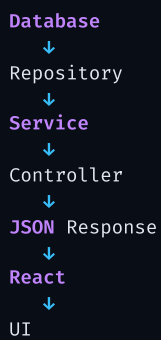
🌟 FULL-STACK JAVASCRIPT SYSTEM TOPOLOGY

USER



Response:

🌟 FULL-STACK JAVASCRIPT SYSTEM TOPOLOGY



39.71 Real-Life Example — Login System

User:

✦ FULL-STACK JAVASCRIPT SYSTEM TOPOLOGY

```
Email
Password
↓
React Login Form
```

Request:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
POST /api/auth/login
```

Node:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Route
↓
Validation
↓
Find User
↓
Compare Password
↓
Generate Token/Session
↓
Response
```

React:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
Response
↓
Store appropriate auth state
↓
Dashboard
```

এখানে Chapter 39-এর অনেক concept একসাথে কাজ করছে:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
HTTP
Async/Await
JSON
Modules
Environment Variables
Error Handling
Database
Authentication
```

39.72 Node.js-এ Promise.all()

একসাথে independent কাজ করতে:

JAVASCRIPT Source

JAVASCRIPT

```
const [users, products] = await Promise.all([
  getUsers(),
  getProducts()
]);
```

Concept:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
graph LR
  getUsers() --- Promise.all()
  getProducts() --- Promise.all()
```

একটি কাজ আরেকটির result-এর উপর নির্ভর না করলে এটি useful।

39.73 Sequential বনাম Parallel

Sequential:

JAVASCRIPT Source

JAVASCRIPT

```
const users = await getUsers();
const products = await getProducts();
```

এখানে দ্বিতীয় কাজ প্রথমটির পরে শুরু হয়। Parallel:

JAVASCRIPT Source

JAVASCRIPT

```
const [users, products] =
  await Promise.all([
    getUsers(),
    getProducts()
  ]);
```

Independent হলে parallel approach latency কমাতে পারে।

39.74 Promise.allSettled()

যদি সব result দেখতে চাও—even কিছু fail করলেও:

JAVASCRIPT Source

JAVASCRIPT

```
const results = await Promise.allSettled([
  getUsers(),
  getProducts(),
  getOrders()
]);
```

এটি প্রতিটি operation-এর:

✦ NODE.JS RUNTIME ARCHITECTURE DIAGRAM

```
fulfilled
rejected
```

status দেয়।

39.75 Node.js-এর জন্য Important JavaScript Concepts

Node.js শেখার আগে এই JavaScript concepts শক্ত হওয়া উচিত:

✦ NODE.JS EVENT LOOP PHASES

- ✓ Variables
- ✓ Objects
- ✓ Arrays
- ✓ Functions
- ✓ Closures
- ✓ Callbacks
- ✓ Promises
- ✓ Async/Await
- ✓ Error Handling
- ✓ Modules
- ✓ Destructuring
- ✓ Spread/Rest
- ✓ Event Loop
- ✓ JSON
- ✓ HTTP

তোমার আগের Chapter-গুলোতে এগুলো already covered হয়েছে।

Part 10 — Must Know Node Pillars, Quick Cheat Sheet, 10 Practice Labs & Final Mental Map

39.76 🔥 MUST KNOW — Node.js-এর জন্য অত্যন্ত গুরুত্বপূর্ণ JavaScript কনসেপ্ট

Node.js Developer হিসেবে অন্তত এগুলো অবশ্যই বুঝবে:

✦ NODE.JS EVENT LOOP PHASES

- 🔥 Node.js Runtime
- 🔥 V8
- 🔥 Event Loop
- 🔥 Non-blocking I/O
- 🔥 process
- 🔥 process.env
- 🔥 CommonJS
- 🔥 ES Modules
- 🔥 fs
- 🔥 path
- 🔥 events
- 🔥 EventEmitter
- 🔥 Buffer
- 🔥 Streams
- 🔥 HTTP
- 🔥 Request / Response
- 🔥 Async/Await
- 🔥 Error Handling
- 🔥 Middleware
- 🔥 REST API
- 🔥 Authentication
- 🔥 Database communication
- 🔥 Environment Variables
- 🔥 Security

JAVASCRIPT Source

JAVASCRIPT

```
// Process
console.log(process.version);

// Environment variable
console.log(process.env.PORT);

// Command line
console.log(process.argv);

// File
import fs from "node:fs/promises";

const data = await fs.readFile(
  "file.txt",
  "utf8"
);

// Path
import path from "node:path";

const file = path.join(
  "uploads",
  "photo.jpg"
);

// OS
import os from "node:os";

console.log(os.cpus().length);

// EventEmitter
import { EventEmitter } from "node:events";

const emitter = new EventEmitter();

emitter.on("login", name => {
  console.log(`${name} logged in`);
});

emitter.emit("login", "Ripon");

// HTTP
import http from "node:http";

const server = http.createServer(
  (req, res) => {
    res.end("Hello");
  }
);

server.listen(3000);
```

Lab 1: CLI Greeting via process.argv

BEGINNER

সমস্যা ও লক্ষ্য: node app.js Ripon কমান্ডের মাধ্যমে টার্মিনাল আর্গুমেন্ট গ্রহণ করে 'Hello Ripon' প্রিন্ট করুন। কোনো নাম না দিলে 'Hello Guest' ফলবাক্য দিন।

PRODUCTION SOLUTION: LAB 1

JAVASCRIPT

```
// app.js
const args = process.argv.slice(2);
const userName = args[0] ?? 'Guest';

console.log(`Hello ${userName}`);

// Verification in terminal:
// $ node app.js Ripon    -> Output: Hello Ripon
// $ node app.js          -> Output: Hello Guest
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** process.argv হলো একটি অ্যারে যার ইনডেক্স ০-তে থাকে নোড পাথ, ১-এ স্ক্রিপ্ট পাথ এবং ২ থেকে ইউজার পাস করা কমান্ড লাইন আর্গুমেন্টস শুরু হয়।

Lab 2: Asynchronous File Reading via fs/promises

BEGINNER

সমস্যা ও লক্ষ্য: message.txt ফাইলটি নন-ব্লকিং অ্যাসিঙ্ক্রোনাস পদ্ধতিতে পড়ে কনসোলে আউটপুট প্রদর্শন করুন এবং এরর হ্যান্ডেল করুন।

PRODUCTION SOLUTION: LAB 2

JAVASCRIPT

```
import fs from 'fs/promises';

async function readMessageFile() {
  try {
    const data = await fs.readFile('message.txt', 'utf-8');
    console.log('File Content: ' + data);
  } catch (err) {
    if (err.code === 'ENOENT') {
      console.error('❌ Error: message.txt file does not exist!');
    } else {
      console.error('❌ Read Error:', err.message);
    }
  }
}

readMessageFile();
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** প্রোডাকশন ব্যাকএন্ডে কখনোই readFileSync ব্যবহার করা উচিত নয়। fs/promises-এর readFile ইভেন্ট লুপকে ফ্রি রেখে ব্যাকগ্রাউন্ড থ্রেডপুলে ফাইল রিড সম্পন্ন করে।

Lab 3: Asynchronous File Writing via fs/promises

BEGINNER

সমস্যা ও লক্ষ্য: output.txt ফাইলে 'Hello from Node.js' টেক্সটটি অ্যাসিঙ্ক্রোনাসভাবে সেভ করুন।

PRODUCTION SOLUTION: LAB 3

JAVASCRIPT

```
import fs from 'fs/promises';

async function writeOutputFile() {
  try {
    const content = 'Hello from Node.js'
    Created on: ' + new Date().toISOString();
    await fs.writeFile('output.txt', content, 'utf-8');
    console.log('✅ File successfully written to output.txt');
  } catch (err) {
    console.error('❌ Write Failed:', err.message);
  }
}

writeOutputFile();
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** fs.writeFile ফাইল না থাকলে স্বয়ংক্রিয়ভাবে নতুন ফাইল তৈরি করে এবং থাকলে পূর্বের কন্টেন্ট ওভাররাইট করে। অ্যাপেন্ড করতে চাইলে { flag: 'a' } অপশন দেওয়া যায়।

Lab 4: High-Performance File Streaming via pipe()

INTERMEDIATE

সমস্যা ও লক্ষ্য: একটি বড় ফাইল (source.txt) থেকে অন্য ফাইলে (destination.txt) মেমোরি লোড না করে স্ট্রিম পাইপলাইনের মাধ্যমে কপি করুন।

PRODUCTION SOLUTION: LAB 4

JAVASCRIPT

```
import fs from 'fs';
import { pipeline } from 'stream/promises';

async function copyFileViaStream() {
  try {
    const sourceStream = fs.createReadStream('source.txt');
    const destStream = fs.createWriteStream('destination.txt');

    // Modern pipeline with auto-cleanup & backpressure management
    await pipeline(sourceStream, destStream);
    console.log('✅ File stream copied successfully without memory leak!');
  } catch (err) {
    console.error('❌ Stream Error:', err.message);
  }
}

copyFileViaStream();
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** বড় সাইজের ফাইল (যেমন 1GB+) একবারেই মেমোরিতে না এনে চাংক আকারে (Chunk by chunk) স্ট্রিম পাইপ করলে সার্ভারে কখনো RAM ক্র্যাশ বা মেমোরি স্পাইক হয় না।

Lab 5: Decoupled Event Architecture via EventEmitter

INTERMEDIATE

সমস্যা ও লক্ষ্য: একটি userRegistered কাস্টম ইভেন্ট তৈরি করুন এবং তাতে লিসেনার অ্যাটাচ করে ইউজার সাইন-আপের সাথে সাথে নোটিফিকেশন লগ প্রিন্ট করুন।

PRODUCTION SOLUTION: LAB 5

JAVASCRIPT

```
import { EventEmitter } from 'events';

class UserService extends EventEmitter {
  register(name, email) {
    console.log(`Processing registration for ${name}...`);
    // Emit event asynchronously or synchronously
    this.emit('userRegistered', { name, email, time: Date.now() });
  }
}

const userService = new UserService();

// Listener 1: Welcome Email
userService.on('userRegistered', user => {
  console.log(`📧 [Email Service]: Sent welcome mail to ${user.email}`);
});

// Listener 2: Activity Logger
userService.on('userRegistered', user => {
  console.log(`📄 [Audit Log]: New user registered: ${user.name}`);
});

userService.register('Ripon', 'ripon@example.com');
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** EventEmitter নোড ব্যাকএন্ডে মডিউলগুলোর মধ্যে লুজ কাপলিং (Loose Coupling) নিশ্চিত করে। কোর রেজিস্ট্রেশন লজিক পরিবর্তন ছাড়াই নতুন লিসেনার যুক্ত করা যায়।

Lab 6: Pure Native Node.js HTTP Web Server

INTERMEDIATE

সমস্যা ও লক্ষ্য: কোনো এক্সট্রানাল প্যাকেজ ছাড়া পিওর http মডিউল দিয়ে পোর্ট ৩০০০-এ একটি সার্ভার তৈরি করুন যা / এ 'Home Page' এবং /about এ 'About Page' রেসপন্স দেয়।

PRODUCTION SOLUTION: LAB 6

JAVASCRIPT

```
import http from 'http';

const server = http.createServer((req, res) => {
  const { url, method } = req;
  res.setHeader('Content-Type', 'text/plain; charset=utf-8');

  if (url === '/' && method === 'GET') {
    res.statusCode = 200;
    res.end('🏠 Welcome to Home Page');
  } else if (url === '/about' && method === 'GET') {
    res.statusCode = 200;
    res.end('📄 Welcome to About Page');
  } else {
    res.statusCode = 404;
    res.end('❌ 404 Route Not Found');
  }
});

const PORT = 3000;
server.listen(PORT, () => {
  console.log(`🚀 Native HTTP Server listening at http://localhost:${PORT}`);
});
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** Express বা Fastify ফ্রেমওয়ার্কগুলো ইন্টারনালি এই নেটিভ http.createServer এর উপর ভিত্তি করেই তৈরি। নেটিভ রিকোয়েস্ট-রেসপন্স লাইফসাইকেল বোঝা অত্যন্ত গুরুত্বপূর্ণ।

Lab 7: RESTful JSON API Endpoint

INTERMEDIATE

সমস্যা ও লক্ষ্য: /api/user রাউটে ক্লায়েন্টকে স্ট্যান্ডার্ড হেডারসহ JSON অবজেক্ট রেসপন্স পাঠান।

PRODUCTION SOLUTION: LAB 7

JAVASCRIPT

```
import http from 'http';

const server = http.createServer((req, res) => {
  if (req.url === '/api/user' && req.method === 'GET') {
    const userPayload = {
      id: 101,
      name: 'Ripon',
      role: 'Full Stack Developer',
      stack: ['JavaScript', 'Node.js', 'React'],
      verified: true
    };

    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(userPayload));
  } else {
    res.writeHead(404, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ error: 'Endpoint Not Found' }));
  }
});

server.listen(3001, () => {
  console.log('🚀 JSON API Server running at http://localhost:3001/api/user');
});
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** JSON রেসপন্স দেওয়ার সময় অবশ্যই 'Content-Type': 'application/json' হেডার সেট করতে হয় এবং JavaScript অবজেক্টকে JSON.stringify() করে পাঠাতে হয়।

Lab 8: Async/Await Data Pipeline with Timeout & Retry

ADVANCED

সমস্যা ও লক্ষ্য: ডাটাবেস বা এক্সটার্নাল API থেকে ডেটা আনার জন্য একটি অ্যাসিনক্রোনাস ফাংশন লিখুন যা এরর হ্যান্ডলিং ও টাইমআউট সাপোর্ট করে।

PRODUCTION SOLUTION: LAB 8

JAVASCRIPT

```
// Simulated DB Call with Promise
function fetchDatabaseRecord(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id > 0) {
        resolve({ id, username: `user_${id}`, status: 'ACTIVE' });
      } else {
        reject(new Error('Invalid record ID provided'));
      }
    }, 150);
  });
}

async function getUserData(id) {
  try {
    console.log(`Fetching record for ID ${id}...`);
    const data = await fetchDatabaseRecord(id);
    console.log('✅ Record Retrieved:', data);
    return data;
  } catch (err) {
    console.error('❌ Fetch Failure:', err.message);
    return null;
  }
}

getUserData(42);
getUserData(-1); // Triggers safe catch block
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** নোড ব্যাকএন্ডে প্রতিটি অ্যাসিনক্রোনাস অপারেশনকে try/catch ব্লকের ভেতরে রাখা জরুরি, যাতে আনহ্যান্ডল্ড রিজেকশন ঘটে সার্ভার প্রসেস ক্র্যাশ না করে।

Lab 9: Parallel API Aggregation via Promise.all()

ADVANCED

সমস্যা ও লক্ষ্য: getUsers(), getProducts(), এবং getOrders() তিনটি স্বাধীন অপারেশনকে প্যারাললে একত্রিকিউট করে একত্রিকিউশন টাইম অপ্টিমাইজ করুন।

PRODUCTION SOLUTION: LAB 9

JAVASCRIPT

```
const fakeFetch = (name, delay) =>
  new Promise(res => setTimeout(() => res(`${name} Data [OK]`), delay));

async function fetchDashboardData() {
  console.time('Parallel Execution Time');

  try {
    // Run all 3 asynchronous calls concurrently in parallel
    const [users, products, orders] = await Promise.all([
      fakeFetch('Users', 200),
      fakeFetch('Products', 300),
      fakeFetch('Orders', 150)
    ]);

    console.log('Aggregation Results:');
    console.log({ users, products, orders });
  } catch (err) {
    console.error('One or more operations failed:', err);
  } finally {
    console.timeEnd('Parallel Execution Time'); // ~300ms total, NOT 650ms!
  }
}

fetchDashboardData();
```

💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** একটির পর একটি await করলে মোট সময় লাগত ২০০+৩০০+১৫০ = ৬৫০ms! Promise.all() ব্যবহার করায় সর্বোচ্চ সময় (~৩০০ms)-এ সবগুলো কল সম্পন্ন হয়।

সমস্যা ও লক্ষ্য: Express/Node.js প্যাটার্ন মেনে ইন-মেমোরি ডেটাসহ একটি সম্পূর্ণ ইউজার ম্যানেজমেন্ট CRUD ব্যাকএন্ড সিস্টেম লিখুন।

PRODUCTION SOLUTION: LAB 10

JAVASCRIPT

```
import http from 'http';

let users = [
  { id: 1, name: 'Ripon', role: 'Admin' },
  { id: 2, name: 'Tanvir', role: 'Editor' }
];

function sendJson(res, status, data) {
  res.writeHead(status, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(data));
}

const server = http.createServer((req, res) => {
  const { method, url } = req;

  // 1. GET /users
  if (method === 'GET' && url === '/users') {
    return sendJson(res, 200, users);
  }

  // 2. GET /users/:id
  if (method === 'GET' && url.startsWith('/users/')) {
    const id = parseInt(url.split('/')[2]);
    const user = users.find(u => u.id === id);
    if (!user) return sendJson(res, 404, { error: 'User not found' });
    return sendJson(res, 200, user);
  }

  // 3. POST /users
  if (method === 'POST' && url === '/users') {
    let body = '';
    req.on('data', chunk => { body += chunk; });
    req.on('end', () => {
      try {
        const { name, role } = JSON.parse(body);
        if (!name) return sendJson(res, 400, { error: 'Name is required' });
        const newUser = { id: users.length + 1, name, role: role || 'User' };
        users.push(newUser);
        return sendJson(res, 201, newUser);
      } catch (err) {
        return sendJson(res, 400, { error: 'Invalid JSON payload' });
      }
    });
  }

  // 4. DELETE /users/:id
  if (method === 'DELETE' && url.startsWith('/users/')) {
    const id = parseInt(url.split('/')[2]);
    const initialLen = users.length;
    users = users.filter(u => u.id !== id);
    if (users.length === initialLen) return sendJson(res, 404, { error: 'User not found' });
    return sendJson(res, 200, { message: `User ${id} deleted successfully` });
  }

  sendJson(res, 404, { error: 'Route not supported' });
});

server.listen(4000, () => {
  console.log('🚀 Complete User CRUD API running at http://localhost:4000/users');
});
```

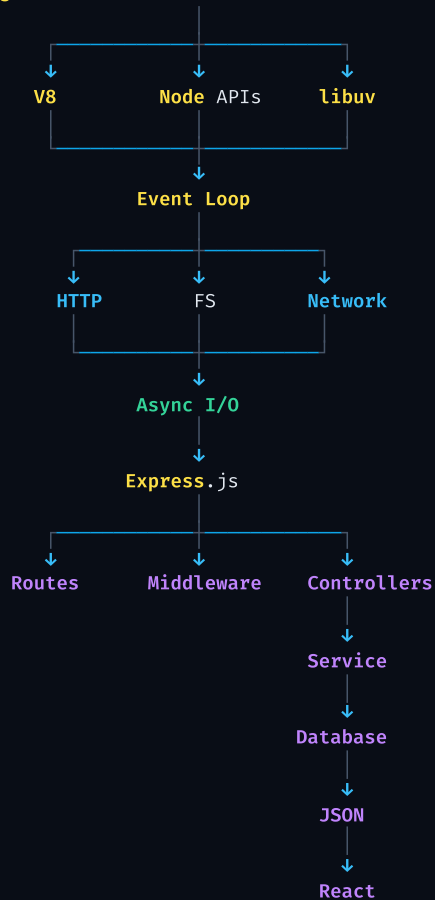
💡 **নোড ব্যাকএন্ড আর্কিটেকচারাল ব্যাখ্যা:** এটি একটি স্বয়ংসম্পূর্ণ RESTful API কন্ট্রোলার যা ইনকামিং রিকোয়েস্ট বডি পাশিং, স্ট্যাটাস কোড হ্যান্ডলিং (200, 201, 400, 404) এবং CRUD অপারেশন সম্পন্ন করে।

39.78 🧠 Final Mental Map — Node.js ব্যাকএন্ড আর্কিটেকচার মানচিত্র

পুরো Chapter 39-এর Node.js ব্যাকএন্ড ইকোসিস্টেম একনজরে:

🌟 NODE.JS ARCHITECTURAL TAXONOMY MAP

Node.js



🔥 এক লাইনে Chapter 39: Node.js হলো JavaScript-এর server-side runtime, যেখানে V8 + Event Loop + Asynchronous I/O + Node APIs ব্যবহার করে scalable backend, REST APIs, file/network processing এবং full-stack application তৈরি করা যায়।

39.79 🔄 Curriculum Progression: Chapter 38 → 39 → 40

🌟 ROADMAP CONTINUITY PIPELINE

Chapter 38: Advanced JavaScript for **React** (Frontend)

↓
Destructuring, Spread, map/filter, Immutability, Closures, Synthetic Events

↓
Chapter 39: Advanced JavaScript for **Node.js** (Backend - THIS CHAPTER)

↓
V8, **libuv**, **Event Loop**, **Buffer**, Streams, **fs/path**, **REST API**, **Express Middleware**

↓
Chapter 40: Real-World Projects, Best Practices & Interview Preparation (Grand Finale)

↓
Production Architecture, Git Workflow, Security Audits, Full-Stack Project & Interview Mastery